TASTES2PLATE · API INTEGRATION GUIDE

T2P Logistics App: Expenses & Cash Ledger API

Integration guide for the logistics app developer

Contents

1. Overview
2. Authentication
3. Conventions (read before coding)
4. How an expense works
5. Data needed when the app starts
6. The expense categories
7. Working with expenses
8. My cash: `GET /v1/cash-ledger/me`
9. Push notifications
10. Endpoint reference (app)
11. Testing checklist
12. Current limitations

This guide covers everything the logistics app needs to let delivery partners, delivery boys, pickup partners and other logistics users:

- record their expenses,
- send them for approval,
- see their own cash balance,
- add money received from the office by uploading the payment screenshot.

The office (admin panel) approves or rejects expenses and records cash handed over in person. The app never does those things.

What's new in version 1.4

#	Change	What the app must do	Section
1	**Add money by screenshot.** When the office sends money by bank or UPI, the person who received it uploads the screenshot. The server reads it and adds the amount to that person's cash balance. New endpoint `POST /v1/ocr/payment-bill`.	Add an **Upload payment screenshot** button on the My cash screen, and refresh the balance after a success.	8.1

Nothing else changes. An app without the new button keeps working.

What's new in version 1.3

#	Change	What the app must do	Section
1	**GST per product** in Food Purchase: every product has its own **GST Yes/No**. Yes = 5% on that product, No = no GST on it. New field `items[].gst_applicable`.	Remove the whole-purchase GST switch and the GST % box. Add a GST Yes/No on each row, and stop sending `gst_applicable` and `gst_percent` at the purchase level.	6.1

#	Change	What the app must do	Section
2	**Products kept together by restaurant.** `food-purchase-lines` now returns restaurants A–Z, with each restaurant's products together.	**Don't sort the lines in the app.** Show them in the order they come.	6.1
3	**Simpler product screen** for the pickup boy: a plain list of Items, Qty and a blank Price, with no restaurant headers. No API change.	Build the screen as shown in step 3. Keep Weight, Discount and Change quantity out of the main list.	6.1

An app still on version 1.2 keeps working, including its GST % box.

What's new in version 1.2

#	Change	What the app must do	Section
1	**Save draft, even half-filled**, in every category. Empty fields are listed in the new `incomplete` field; Submit is refused (422) until they are filled.	Add a **Save draft** button to every form. Show an **"Incomplete"** label on such drafts and highlight the missing fields.	4.1, 7.1, 7.4, 7.5
2	**GST Yes/No** for the whole Food Purchase (default Yes), with a **GST %** box (default 5, any value above 0 up to 28). New fields `gst_applicable`, `gst_percent`.	Add the switch and the % box to the Food Purchase form; use the % in the on-screen totals.	6.1
3	**Food Purchase order list:** orders delivering **tomorrow or the day after**, plus orders whose purchase is filled but **not approved yet**. Approved ones disappear. Each order has a new `expense` field.	Don't let the user tick an order whose `expense` is not `null`; open that expense instead.	6.1
4	**Delivery order list:** orders delivering **today or tomorrow**, plus orders whose delivery expense is filled but **not approved yet**. Approved ones disappear. New fields `delivery_date`, `timeslot`, `expenses`.	Show the list as it comes; no date filtering in the app.	6.3
5	**Search (`q`)** in both order lists only searches inside that list; older orders are not found any more.	Nothing.	6.1, 6.3
6	The office (main admin) can **delete** an expense. It disappears from the user's list, any cash taken is put back, and its order comes back to the order list.	Treat a 404 on an expense the user had open as "deleted"; refresh the list.	7.5, 8

1. Overview

1.1 What the app needs to build

Screen	What it does	Main API calls
Expenses home	My expenses with status tabs (Draft, Pending approval, Approved, Rejected, Cancelled), an "Incomplete" label on half-filled drafts, and my cash balance	`GET /v1/expenses`, `GET /v1/cash-ledger/me`
New expense: choose type	Seven categories	`GET /v1/expenses/meta`

Screen	What it does	Main API calls
Food Purchase form	Tick orders, then a simple list of Items, Qty, Price and a GST Yes/No on each row; weight, discount and quantity changes behind a "More" option	`food-purchase-orders`, `food-purchase-lines`, `POST /v1/expenses`
Intercity Logistics form	From city (locked), To city, rate per kg, weight	`cities`, `POST /v1/expenses`
Delivery form	Order, default distance (shown, read-only), actual distance	`lookups/orders`, `default-distance`, `POST /v1/expenses`
Miscellaneous form	Expense head or Other, details, reason, amount	`expense-heads`, `POST /v1/expenses`
Petrol form	Rate per litre, litres, start km reading, vehicle, pump	`POST /v1/expenses`
Travelling form	From, to, travel mode, date, purpose, amount	`POST /v1/expenses`
Cancelled Orders	What happened to the food of a cancelled order: adjusted on another order, consumed or scrapped; partner approves it	`lookups/cancelled-orders`, `POST /v1/expenses`, `approve`
Expense detail	Details, photos, history, rejection reason, missing fields of an incomplete draft; Edit, Submit, Move to draft, Cancel buttons	`GET /v1/expenses/:id`, actions
My cash	Balance, period summary, list of entries, Upload payment screenshot button	`GET /v1/cash-ledger/me`, `POST /v1/ocr/payment-bill`

1.2 Base URL

Environment	Base URL
Production	`https://webapi.tastes2plate.com` (available once T2P deploys the expense module)
Test	Ask T2P for the test server URL

Every expense endpoint starts with `/v1`. The login endpoints are the existing `/app/...` ones.

1.3 Who can use it

Users with `user_type` delivery_boy, delivery_partner, pickup_boy, pickup_partner, cargo_partner, lp_head or lp_manager. Each user only ever sees **their own** expenses and cash ledger.

2. Authentication

2.1 Login (existing OTP flow, now returns a token)

Step 1: request OTP (unchanged)

```
POST /app/logistic-login
Content-Type: application/json
```

```
{ "mobile": "9800000010" }
```

> **Note:** this response currently also contains the OTP. Do **not** use it. It will be removed; always ask the user to type the OTP they received by SMS.

Step 2: verify OTP and get the token

```
POST /app/verify-logistic
Content-Type: application/json

{ "mobile": "9800000010", "otp": "482913" }
```

```
{
  "status": "success",
  "message": "Otp verified",
  "result": {
    "_id": "605dc565d5eaba0008a50e4e",
    "full_name": "Ravi Kumar",
    "user_type": "delivery_partner",
    "...": "..."
  },
  "office_data": { "_id": "60ef0e96d4473c18f88872f5", "name": "KOLKATA OFFICE", "...": "..." },
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...."
}
```

Only `token` is new; the other fields are the same as before.

2.2 Using the token

- Save `token` in secure storage (Keychain on iOS, EncryptedSharedPreferences/Keystore on Android).
- Send it on **every** `/v1` request:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9....
```

- The token is valid for **30 days**.
- On HTTP **401**, clear the token and show the OTP login again.
- The server identifies the user from the token. **Never send a user id** to say who the user is; it is ignored or refused.

```
// 401 example
{ "status": "error", "message": "Login required" }
```

3. Conventions (read before coding)

3.1 Money

Direction	Format	Example
App → server	Rupees, as a string or number, at most 2 decimals	`"250.50"`, `"80"`
Server → app	**Paise**, as an integer	`25050` means ₹250.50

Display rule: `rupees = paise / 100`, shown with 2 decimals.

The server calculates every total. Anything you calculate on the phone is only a preview; always show the server's numbers after saving.

3.2 Quantities and precision

Value	Unit	Decimals allowed
Food quantity, weight	count, kg	3
Litres, distance	litres, km	2
Start and end km readings	km	1
Money	rupees	2

Negative numbers are refused everywhere, except the signed quantity in Food Purchase adjustments (section 6.1).

3.3 Dates

- Send dates as `"YYYY-MM-DD"` (a calendar day in India time), or a full ISO date-time such as `"2026-10-10T09:30:00+05:30"`.
- No future dates.
- The app can backdate at most **7 days** (`meta.backdate_days`).
- If you leave `expense_date` out, it is "now".

3.4 Response format

Every response is JSON with this envelope:

```
{ "status": "success", "message": "Expense saved", "data": {} }
```

HTTP	`status`	Meaning	What the app should do
200 / 201	`success`	OK	Use `data`
401	`error`	Not logged in or token expired	Go to OTP login
403	`error`	Not allowed (e.g. the order isn't assigned to this user)	Show `message`
404	`error`	Not found, or it belongs to someone else	Show `message`, go back

HTTP	`status`	Meaning	What the app should do
409	`error`	Conflict: already exists, or changed since you loaded it	Show `message`, reload the record
422	`validation_error`	Some fields are wrong; see `errors[]`	Show each error next to its field
429	`error`	Too many requests (more than 60 changes per minute)	Wait and retry
500	`error`	Server problem	Show "Try again later"

Validation errors:

```
{
  "status": "validation_error",
  "message": "Please correct the highlighted fields",
  "errors": [
    {
      "field": "details.rate_per_litre",
      "message": "rate_per_litre must be a positive number with at most 2 decimal places"
    },
    {
      "field": "details.litres",
      "message": "litres must be a positive number with at most 2 decimal places"
    },
    { "field": "details.end_km", "message": "End km can't be lower than start km" }
  ]
}
```

`field` uses the same path as your request body. Category fields are always under `details.`, for example `details.items[1].rate` or `details.items[0].adjustments[0].note`. Other fields: `expense_date`, `files`, `category`, `reason`.

3.5 Safe retries (important on mobile networks)

When creating an expense, generate a random `client_request_id` (a UUID) **once per new expense form** and send it with the create request.

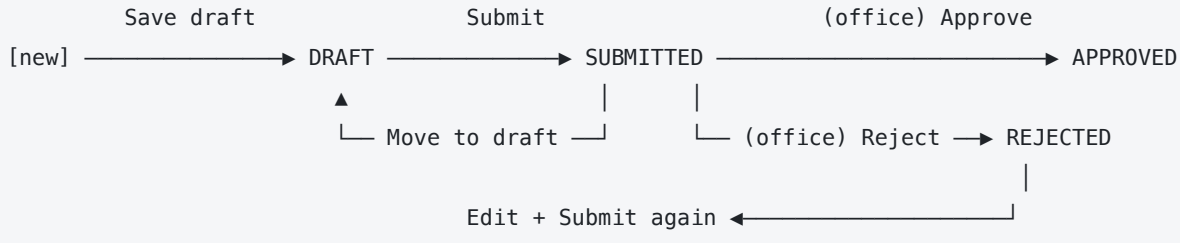
- If the connection drops and you send the same request again with the same id, the server returns the **already saved** expense (HTTP 200, message "This expense was already saved") instead of creating a duplicate.
- Generate a new id only when the user starts a new expense.

3.6 Versions (optional for the app)

Every expense has a `version` number that goes up on each change. You may send `"version"` with edit, submit, move-to-draft and cancel. If someone else changed the expense in between, you get **409**; reload and show the fresh data. If you don't send it, the latest version is used.

4. How an expense works

4.1 Status flow



Status	Shown to user as	User can
`draft`	Draft	Edit, add or remove photos, Submit, Cancel
`submitted`	Pending approval	Move to draft (to edit), Cancel
`approved`	Approved	Only view
`rejected`	Rejected (show `review.rejection_reason`)	Edit, add or remove photos, Submit again, Cancel
`cancelled`	Cancelled	Only view

Save draft (every category): every form should have two buttons, **Save draft** and **Submit**. The user can save a form half-filled and finish it later, e.g. save the purchase now and type the rates after matching the bill. Create without `submit` (or edit with `PATCH`) and leave fields empty: the expense is saved as a draft and its **`incomplete`** list names what is still missing:

```
"incomplete": [
  { "field": "details.items[0].rate", "message": "items[0].rate is required" },
  { "field": "details.items[1].rate", "message": "items[1].rate is required" }
]
```

- Show such drafts with an "Incomplete" label (the list `GET /v1/expenses` also returns `incomplete`) and highlight the fields when the user opens it.
- Only **empty** fields are allowed in a draft. A wrong value (letters in a number, a negative amount, GST above 28 %) is still refused with **422**.
- The order is always required for Food Purchase, Delivery and Cancelled Orders, even in a draft.
- **Submit** (or Approve for Cancelled Orders) answers **422** with the same `errors` list while anything is missing: `"message": "This draft is not complete. Fill in the missing details, then try again."`
- A draft never touches the cash balance; cash is deducted only on Submit.

Cancelled Orders are different: there is no Submit and no office approval. The partner/boy saves a draft, can edit it, and taps **Approve** when the record is complete (`POST /v1/expenses/:id/approve`). Approved is final. See section 6.7.

Don't hard-code these rules. Every expense response contains an `actions` object; show a button only when its value is `true`:

```
"actions": {
  "edit": true, "upload": true, "submit": true, "recall": false,
  "approve": false, "reject": false, "cancel": true, "delete": false
}
```

(`approve` and `reject` are always `false` for app users.)

4.2 Effect on the user's cash

What happens	Cash ledger
Save as draft	No change
Submit (or submit again)	**Amount is deducted immediately** (entry type `EXPENSE_DEBIT`)
Office approves	No change (it was already deducted)
Office rejects or deletes / user cancels / user moves it back to draft	**Amount is put back** (`EXPENSE_REVERSAL`)

The submit response includes the new balance (`balance_after` , in paise). It also includes a `warning` if the balance is now negative; show that warning to the user.

5. Data needed when the app starts

5.1 `GET /v1/expenses/meta`

Call it when the user opens the Expenses section. Cache it for the session.

```
{
  "status": "success",
  "data": {
    "me": {
      "_id": "605dc565d5eaba0008a50e4e",
      "name": "Ravi Kumar",
      "mobile": "9800000010",
      "user_type": "delivery_partner",
      "is_staff": false,
      "cities": [ "6040eae14a4b6c0008fe1aff" ],
      "default_city": "6040eae14a4b6c0008fe1aff",
      "default_office": "60ef0e96d4473c18f88872f5"
    },
    "permissions": {
      "expense": { "add_edit": true, "approve": false, "...": "..." },
      "ledger": { "...": "..." }
    },
    "categories": [
      {
        "code": "FOOD_PURCHASE",
```

```
        "label": "Food Purchase",
        "document_required": false,
        "fields": [ "..." ]
      },
      {
        "code": "INTERCITY_LOGISTICS",
        "label": "Intercity Logistics",
        "document_required": false,
        "fields": [ "..." ]
      },
      { "code": "DELIVERY", "label": "Delivery/purchase/Other travel distance", "...": "..." },
      { "code": "MISC", "label": "Miscellaneous", "...": "..." },
      { "code": "PETROL", "label": "Petrol Filling", "...": "..." },
      { "code": "TRAVEL", "label": "Travelling", "...": "..." }
    ],
    "statuses": [ "draft", "submitted", "approved", "rejected", "cancelled" ],
    "travel_modes": [
      { "value": "bus", "label": "Bus" },
      { "value": "train", "label": "Train" },
      { "value": "taxi", "label": "Taxi" },
      { "value": "auto_rickshaw", "label": "Auto-rickshaw" },
      { "value": "bike", "label": "Bike" },
      { "value": "flight", "label": "Flight" },
      { "value": "other", "label": "Other" }
    ],
    "expense_heads": [ { "_id": "6ac9...01", "name": "Packaging material", "active": true } ],
    "gst_rate_percent": 5,
    "backdate_days": 7,
    "upload": { "max_files": 10, "max_mb": 10, "allowed": "JPG, PNG, WEBP, HEIC or PDF" },
    "ledger_types": {
      "OPENING_BALANCE": "Opening Balance",
      "CASH_RECEIVED_HO": "Cash Received from Head Office",
      "EXPENSE_DEBIT": "Expense",
      "EXPENSE_REVERSAL": "Expense Reversal",
      "ADJUSTMENT_CREDIT": "Manual Credit Adjustment",
      "ADJUSTMENT_DEBIT": "Manual Debit Adjustment",
      "DUTY_DEBIT": "Duty (km x rate)",
      "DUTY_REVERSAL": "Duty Correction"
    }
  }
}
```


Use `categories[].label`, `travel_modes`, `expense_heads`, `gst_rate_percent`, `backdate_days` and `upload` from here instead of hard-coding them; the office can change them without an app release.

5.2 Other lists

Call	Returns
`GET /v1/expenses/lookups/cities`	`[{ "_id", "name", "ps" }]`, all active cities
`GET /v1/expenses/lookups/expense-heads`	`[{ "_id", "name", "active" }]`, heads for Miscellaneous

6. The expense categories

All of them are created with the same call, `POST /v1/expenses`; only `category` and `details` differ (section 7.1). The examples below show only the `details` object.

6.1 Food Purchase (`FOOD_PURCHASE`)

The pickup partner/boy buys food at the restaurant for one or more orders.

Rules

- **Which orders:** only orders where this user is the order's **pickup partner or pickup boy**:
  - orders whose **delivery date is tomorrow or the day after** (India time), and
  - orders whose food purchase is **filled but not approved yet** (draft, submitted or rejected), whatever their date.

  Once the office **approves** the purchase, the order disappears from the list. If the office **deletes** a purchase, its order comes back (while its delivery date is still tomorrow / the day after). Other orders are never listed; `q` (order number) only searches within this list.
- `expense` **on each order:** `null` = not bought yet, the user can tick it. Otherwise `{ "_id", "expense_no", "status" }` = it already has a purchase: **don't let the user tick it**; open that expense instead (e.g. to fix a rejected one).
- **Several orders in one purchase:** allowed (up to 50), but all must be from the **same city**.
- **Same product, one line:** if a product is in several orders, it appears **once**, with the quantities added up. Example: order A has Chicken Biryani ×2 and order B has Chicken Biryani ×3, so the line is Chicken Biryani with order quantity **5**.
- **Quantity can't be edited:** the **order quantity is fixed** and comes from the orders. **Don't send a quantity**; the server ignores it.
- **Changing the quantity:** if the user bought more or fewer, they add a **quantity change** (`adjustments[]`) with:
  - a signed quantity (`"+2"` or `"-1"`),
  - an optional Order ID it is for,
  - a **required reason** (`note`).

  Final quantity = order quantity + all changes.
- **Products can't be removed:** if a product wasn't bought, reduce it to 0 with a change and a reason. Then no rate is needed.
- **One purchase per order:** an order can be in only one active food purchase. Cancelling the purchase (or the office deleting it) frees its orders.

- **Kept together by restaurant:** lines come with restaurants A–Z and each restaurant's products together. **Keep this order; don't sort in the app.**
- **GST per product:** each row has a **GST Yes/No** switch, **Yes by default**, because the user checks it against the bill. Yes = **5%** on that product (`gst_rate_percent` from meta), No = no GST on it. Changing one row doesn't change any other. There is no % box. Send `items[].gst_applicable` (`true` / `false`). If you leave it out, a new product has GST Yes at 5%, and a product in an edited purchase keeps its current GST.
- **Totals:** for each product, Subtotal = rate × final quantity; Taxable = subtotal – discount; GST = 5% of taxable, rounded to the paisa (0 when GST is No); Total = taxable + GST. The server recalculates and its figures are final.

Screen flow

1. `GET /v1/expenses/lookups/food-purchase-orders?q=T2P-179` shows the list of orders with checkboxes. `q` (search by order number) is optional.


```
{
  "status": "success",
  "data": [
    {
      "_id": "6ac33f67cd32c8f0c6301fa6",
      "orderid": "T2P-1790000101",
      "status": "vendor_approved",
      "created_date": "2026-10-09T11:10:22.917Z",
      "delivery_date": "2026-10-11T00:00:00.000Z",
      "timeslot": "After Noon (12:00 PM - 04:00 PM)",
      "vendor_city": { "_id": "6040eae14a4b6c0008fe1aff", "name": "KOLKATA" },
      "order_city": { "_id": "608f769984eaa700091774fe", "name": "PUNE" },
      "pickup_partner": { "_id": "605dc565d5eaba0008a50e4e", "name": "INTERNAL_KOLKATA" },
      "pickup_boy": null,
      "expense": null,
      "products": [
        {
          "line_id": "6ac33f67cd32c8f0c6301fa7",
          "product": "65ec39202df3d12ed065251f",
          "product_name": "Chicken Biryani - Example Restaurant, Kolkata",
          "quantity": 2,
          "vendor": { "_id": "65ec0f797c7e2fa1f658103f", "name": "Example Restaurant" },
          "unit_weight_kg": 0.55
        }
      ]
    }
  ]
}
```


2. When the user has ticked orders, call **`GET /v1/expenses/lookups/food-purchase-lines?orders=<id1>,<id2>`**. It returns one line per product, already combined. **Show the lines in this order**: validation errors refer to `details.items[i]` by this index.

```
{
  "status": "success",
  "data": {
    "orders": [
      {
        "_id": "6ac33f67cd32c8f0c6301fa6",
        "orderid": "T2P-1790000101",
        "status": "vendor_approved"
      },
      {
        "_id": "6ac25d53cd32c8f0c62d7a74",
        "orderid": "T2P-1790000102",
        "status": "vendor_approved"
      }
    ],
    "vendor_city": "6040eae14a4b6c0008fe1aff",
    "lines": [
      {
        "product": "65ec39202df3d12ed065251f",
        "product_name": "Chicken Biryani - Example Restaurant, Kolkata",
        "vendor": { "_id": "65ec0f797c7e2fa1f658103f", "name": "Example Restaurant" },
        "orders": [
          { "order": "6ac33f67cd32c8f0c6301fa6", "orderid": "T2P-1790000101", "quantity": 2 },
          { "order": "6ac25d53cd32c8f0c62d7a74", "orderid": "T2P-1790000102", "quantity": 3 }
        ],
        "order_quantity": 5,
        "unit_weight_kg": 0.55
      },
      {
        "product": "60b9b6f57e8bd40009624507",
        "product_name": "Rasgulla 1 kg - Example Sweets, Kolkata",
        "vendor": { "_id": "60b895375df2b80008ba4f8e", "name": "Example Sweets" },
        "orders": [
          { "order": "6ac25d53cd32c8f0c62d7a74", "orderid": "T2P-1790000102", "quantity": 1 }
        ],
        "order_quantity": 1,
        "unit_weight_kg": 1
      }
    ]
  }
}
```


3. **Show a simple list.** The pickup boy only has to type the price:

Items	Qty	Price (per item)	GST	Total
Chicken Biryani - Example Restaurant, Kolkata	5	*blank*	[Yes]	—
Rasgulla 1 kg - Example Sweets, Kolkata	1	*blank*	[No]	—

Below the list: Subtotal, GST and Grand total.

- **Items:** `product_name`. It already includes the restaurant, so there are no restaurant headers.
- **Qty:** `order_quantity`, read-only. After a quantity change, show the final quantity instead.
- **Price:** always **blank** at the start. It is sent as `rate`, the price of **one** item. Never show the product's selling price or purchase price here.
- **GST:** a Yes/No switch on the row, Yes by default. It is sent as `gst_applicable`.
- **Total:** filled in as the price is typed, including this row's GST, so a wrong entry is easy to spot.
- **Save draft** works with prices still blank, so the user can fill them in later.

Put these behind a **"More"** option on each row, not in the main list:

- the orders the product came from;
- **Change quantity**, which opens Add/Less, quantity, Order ID (optional) and Reason (required);
- **Weight (kg)**, optional (you may pre-fill `unit_weight_kg × order_quantity`);
- **Discount** (₹ or %), optional.

4. **Request `details`:**

```
{
  "orders": [ "6ac33f67cd32c8f0c6301fa6", "6ac25d53cd32c8f0c62d7a74" ],
  "items": [
    {
      "product": "65ec39202df3d12ed065251f",
      "rate": "250",
      "gst_applicable": true,
      "weight_kg": "2.2",
      "discount_type": "amount",
      "discount_value": "20",
      "adjustments": [
        {
          "quantity": "-1",
          "orderid": "T2P-1790000102",
          "note": "Only 4 available at the shop"
        }
      ]
    },
    {
      "product": "60b9b6f57e8bd40009624507",
      "rate": "180",
      "weight_kg": "1",
      "gst_applicable": false
    }
  ]
}
```

Field	Required	Notes
`orders`	yes	Ids of the ticked orders (1–50, same city)
`items[]`	yes	**One item for every line** returned by `food-purchase-lines`

Field	Required	Notes
`items[].product`	yes	From the line
`items[].rate`	yes, if final quantity > 0	Rupees per unit
`items[].gst_applicable`	no	`true` (default) = 5% on this product; `false` = no GST
`items[].weight_kg`	no	Total weight for this line
`items[].discount_type`	no	`"amount"` (default) or `"percent"`
`items[].discount_value`	no	Rupees for `amount`, percent for `percent`; cannot exceed the subtotal
`items[].adjustments[]`	no	Up to 20 per line
`adjustments[].quantity`	yes	Signed, not 0: `"+2"`, `"-1"`, `"-0.5"`
`adjustments[].orderid`	no	An existing order number such as `"T2P-1790000102"`
`adjustments[].note`	yes	At least 3 characters: why the quantity changed

In a draft you may leave `items[].rate`, `adjustments[].quantity` and `adjustments[].note` empty; they are then listed in `incomplete`. `orders` and one item per line are always needed.

Result: Chicken Biryani has order quantity 5 and a change of −1, so the final quantity is 4.

- Subtotal ₹1,000.00, discount ₹20.00, taxable ₹980.00, GST at 5% ₹49.00, **total ₹1,029.00**.
- Rasgulla, GST No: subtotal ₹180.00, GST ₹0, **total ₹180.00**.
- **Expense total ₹1,209.00** (`120900` paise).

Each saved item in the response looks like this:


```
{
  "product": "65ec39202df3d12ed065251f",
  "product_name": "Chicken Biryani – Example Restaurant, Kolkata",
  "vendor": { "_id": "65ec0f797c7e2fa1f658103f", "name": "Example Restaurant" },
  "orders": [
    {
      "order": "6ac33f67cd32c8f0c6301fa6",
      "orderid": "T2P-1790000101",
      "line_id": "6ac33f67cd32c8f0c6301fa7",
      "quantity": 2
    },
    {
      "order": "6ac25d53cd32c8f0c62d7a74",
      "orderid": "T2P-1790000102",
      "line_id": "6ac25d0ccd32c8f0c62d7870",
      "quantity": 3
    }
  ],
  "order_quantity": 5,
  "adjustments": [
    {
```

```
      "quantity": -1,
      "orderid": "T2P-1790000102",
      "order": "6ac25d53cd32c8f0c62d7a74",
      "note": "Only 4 available at the shop"
    }
  ],
  "quantity": 4,
  "rate": 25000,
  "weight_kg": 2.2,
  "discount_type": "amount",
  "discount_value": 2000,
  "subtotal": 100000,
  "discount": 2000,
  "taxable": 98000,
  "gst_rate_bp": 500,
  "gst": 4900,
  "total": 102900
}
```

(`gst_rate_bp` is basis points: 500 = 5%. For a percent discount, `discount_value` is also in basis points, e.g. 1250 = 12.5%.) In an incomplete draft, an item without a rate has `"rate": null` and its money fields are 0.

When opening a purchase for editing: fill each row's switch from its `gst_rate_bp` (`0` = No, above 0 = Yes), and show its amount from `gst`. `details.gst_applicable` and `details.gst_rate_bp` are still returned as a summary for older screens; don't use them.

Possible errors

- **409**, an order is already in a purchase: `"T2P-1790000102 already has a food purchase (FP-2627-000001)."`
- **403**, an order is not assigned to this user: `"Order T2P-1790000101 is not assigned to you for pickup."`
- **422**, orders from different cities: field `details.orders`.
- **422**, a quantity change has no reason: field `details.items[0].adjustments[0].note`.
- **422**, the final quantity is below 0: field `details.items[0].adjustments`.

6.2 Intercity Logistics (`INTERCITY_LOGISTICS`)

Cost of sending food from the partner's city to another city.

```
{
  "from_city": "6040eae14a4b6c0008fe1aff",
  "to_city": "6040eb2b4a4b6c0008fe1b01",
  "rate_per_kg": "38.50",
  "weight_kg": "12.5",
  "reference_no": "AWB 123-4567"
}
```

- `from_city`: **pre-fill with the user's city** (`meta.me.default_city`) and lock it. It must be one of `meta.me.cities`.

- `to_city` : from `lookups/cities` ; must be different from `from_city` .
- `reference_no` (AWB / challan number) is optional.
- In a draft, every field may be left empty.
- **Total = rate per kg × weight.** Here 38.50 × 12.5 = ₹481.25 (`48125`).

6.3 Delivery (`DELIVERY`)

Records the distance travelled for one order.

1. `GET /v1/expenses/lookups/orders?q=` lists orders where this user is the pickup or delivery partner/boy. It shows orders whose **delivery date is today or tomorrow** (India time), plus orders where this user's delivery expense is **filled but not approved yet**. When the user's expense is **approved** (for every side they are on), the order disappears. Other orders are never listed; `q` only searches within this list. Each order also has `delivery_date` , `timeslot` and `expenses` (this user's delivery expenses for it, with `status` and `side`). Each order has a `side` (`pickup` or `delivery`) and `sides` , all the sides this user is assigned to. If `sides` has both, let the user choose and send `side` .


```
{
  "status": "success",
  "data": [
    {
      "_id": "6ac36639cd32c8f0c630e184",
      "orderid": "T2P-1790000103",
      "status": "delivery_boy_started",
      "created_date": "2026-10-21T08:12:00.000Z",
      "delivery_date": "2026-10-23T00:00:00.000Z",
      "timeslot": "Evening (04:00 PM - 08:00 PM)",
      "vendor_city": { "_id": "6040eae14a4b6c0008fe1aff", "name": "KOLKATA" },
      "order_city": { "_id": "6040eb2b4a4b6c0008fe1b01", "name": "DELHI" },
      "pickup_partner": { "_id": "605dc565d5eaba0008a50e4e", "name": "INTERNAL_KOLKATA" },
      "pickup_boy": null,
      "delivery_partner": { "_id": "62035708dab0fe7899fcd088", "name": "INTERNAL_DELHI" },
      "delivery_boy": null,
      "side": "delivery",
      "sides": [ "delivery" ],
      "expenses": [
        {
          "_id": "6ad1f31e50f24b0f15f6fddf",
          "expense_no": "DL-2627-000004",
          "status": "draft",
          "side": "delivery"
        }
      ]
    }
  ]
}
```


If `expenses` already has an expense for the side the user wants, open it instead of creating a new one (a second one gives 409).

2. `GET /v1/expenses/lookups/orders/<orderId>/default-distance?side=delivery` returns the default distance to show (read-only). `side` is only needed when the user is on both sides:
    - **pickup:** office → restaurant;
    - **delivery:** office → customer.


```
{
  "status": "success",
  "data": {
    "order": "6ac36639cd32c8f0c630e184",
    "orderid": "T2P-1790000103",
    "side": "delivery",
    "km": 12.7,
    "source": "google_road",
    "message": null
  }
}
```


`source` is `google_road` (Google Maps) or `straight_line`. If a location is missing, `km` is `null` and `message` explains why, for example "Default distance not available: the restaurant has no location saved.". Still let the user save; show the message.

3. **Request `details`:**


```
{ "order": "6ac36639cd32c8f0c630e184", "actual_distance_km": "14.2", "side": "delivery" }
```


- The server stores `default_distance_km`, `actual_distance_km` and **`deviation_km = actual − default`** (it can be negative).
- **The amount is ₹0 for now**, because the payment rule for delivery isn't decided yet. Delivery expenses therefore never change the cash balance.
- `side` is optional; send it only when the user is on both sides of the order. A side the user is not assigned to gives **403**.
- One delivery expense per order per person per side; a second one gives **409**.
- In a draft, `actual_distance_km` may be left empty (listed in `incomplete`); `order` is always needed.

6.4 Miscellaneous (`MISC`)


```
{
  "expense_head": "6ac9...01",
  "description": "2 rolls of packing tape",
  "reason": "Packing boxes for Delhi",
  "amount": "85"
}
```


or, with a head that isn't in the list:


```
{
  "head_other": "Packing tape",
  "description": "2 rolls of packing tape",
  "reason": "Packing boxes for Delhi",
```

```
  "amount": "85"
}
```

- Show `meta.expense_heads` plus an **"Other"** option. Other needs `head_other` (free text, up to 100 characters).
- `description` ("Details") and `reason` are required, up to 1,000 characters each.
- `amount` is the total.
- In a draft, every field may be left empty.

6.5 Petrol Filling (`PETROL`)

```
{
  "rate_per_litre": "104.77",
  "litres": "3.25",
  "start_km": "15230",
  "end_km": "15310",
  "filled_at": "2026-10-10T09:15:00+05:30",
  "vehicle_no": "WB02AB1234",
  "pump_name": "HP, Salt Lake"
}
```

- **Total = rate × litres.** Here 104.77 × 3.25 = ₹340.50 (`34050`).
- `start_km` and `end_km` are odometer readings, both **optional**, with up to 1 decimal. If both are sent, `end_km` can't be lower than `start_km` .
- `filled_at` is optional (defaults to `expense_date`); `vehicle_no` and `pump_name` are optional.
- **Several petrol entries per day are allowed.**
- If `start_km` is sent and is lower than the user's previous petrol entry, the expense still saves but is **flagged** for the office (see `flags`).
- In a draft, every field may be left empty.

6.6 Travelling (`TRAVEL`)

```
{
  "from_place": "Howrah",
  "to_place": "Sealdah",
  "travel_mode": "taxi",
  "journey_date": "2026-10-10",
  "purpose": "Collect cash from office",
  "amount": "320"
}
```

- `travel_mode` is one of `meta.travel_modes` . If it is `"other"` , `travel_mode_other` is required (for example `"Ferry"`).
- `journey_date` is optional (defaults to `expense_date`); `purpose` is optional; `amount` is the total.
- In a draft, every field may be left empty.

6.7 Cancelled Orders (`CANCELLED_ORDER`)

Records what happened to the food of an order that was **cancelled** (status `cancel`) or **refused by the customer** (`rejected_customer`). There is no money: `amount` is 0 and the cash balance never changes.

Who sees which cancelled order. It depends on where the order was just before it was cancelled:

Order was at (status before cancel)	Shown to
Picked up (`pickup_boy_started`)	**Pickup city**: the order's pickup partner / pickup boy
Received at origin airport (`delivered_to_cargo_partner`), cargo loaded to flight (`cargo_delivery_started`), cargo off loaded (`cargo_off_loaded`), received at destination airport (`received_destination_airport`), received by delivery partner (`delivered_to_delivery_partner`), delivery boy assigned (`delivery_boy_assigned`), out for delivery (`delivery_boy_started`)	**Destination city**: the order's delivery partner / delivery boy
Anything earlier (vendor approved, pickup boy assigned, …)	Nobody: the food was never picked up

The server works this out from the order's history; the app only calls the list.

Screen flow

1. `GET /v1/expenses/lookups/cancelled-orders?q=` lists the cancelled orders this user has to handle (last 45 days), excluding orders already recorded.


```
{
  "status": "success",
  "data": [
    {
      "_id": "6ac36639cd32c8f0c630e1a1",
      "orderid": "T2P-1790000301",
      "status": "cancel",
      "status_label": "Cancelled",
      "stage_status": "pickup_boy_started",
      "stage_label": "Picked up",
      "side": "pickup",
      "city": { "_id": "6040eae14a4b6c0008fe1aff", "name": "KOLKATA" },
      "vendor_city": { "_id": "6040eae14a4b6c0008fe1aff", "name": "KOLKATA" },
      "order_city": { "_id": "6040eb2b4a4b6c0008fe1b01", "name": "DELHI" },
      "created_date": "2026-10-14T09:20:00.000Z",
      "products": [
        {
          "line_id": "6ac36639cd32c8f0c630e1a2",
          "product": "65ec39202df3d12ed065251f",
          "product_name": "Chicken Biryani - Example Restaurant, Kolkata",
          "quantity": 3,
          "vendor": { "_id": "65ec0f797c7e2fa1f658103f", "name": "Example Restaurant" }
        }
      ]
```

```
    }
  ]
}
```

2. For each product show the quantity (fixed) and let the user add entries. Each entry is one of:

`type`	Meaning	Fields
`adjusted`	Adjusted on an order: the food was used for another order	`quantity`, `date`, `orderid` (**required**, an existing order number)
`consumed`	Eaten	`quantity`, `date`, `note` (**required**: who consumed it)
`scrapped`	Spoiled / thrown away	`quantity`, `date`, `note` (optional: why)

Show **Left = quantity – all entries** for every product. The total of the entries cannot be more than the quantity. Left may stay above 0.

3. **Request `details`:**

```
{
  "order": "6ac36639cd32c8f0c630e1a1",
  "cancel_reason": "Customer cancelled after pickup",
  "items": [
    {
      "line_id": "6ac36639cd32c8f0c630e1a2",
      "dispositions": [
        {
          "type": "adjusted",
          "quantity": "2",
          "date": "2026-10-15",
          "orderid": "T2P-1790000310"
        },
        {
          "type": "consumed",
          "quantity": "1",
          "date": "2026-10-15",
          "note": "Eaten by office staff"
        }
      ]
    }
  ]
}
```

Field	Required	Notes
`order`	yes	From the list
`cancel_reason`	no	Up to 500 characters
`items[].line_id`	yes	From `products[].line_id`
`items[].dispositions[]`	no	Up to 30 per product

Field	Required	Notes
`dispositions[].date`	no	`YYYY-MM-DD` ; default today; at most 7 days back

Each saved item in the response has `quantity` , `dispositions[]` and `left` . For an `adjusted` entry, the server also stores the order id it found, and `product_mismatch: true` if that order does not contain the same product (the record still saves; the office sees a warning). `details.total_left` is the sum of `left` .

4. **Save, edit, approve**
    - Create with `POST /v1/expenses` (`category: "CANCELLED_ORDER"`); it is saved as a **draft**. Do not send `submit` .
    - Add more entries later with `PATCH /v1/expenses/:id` , sending all `items` again.
    - When done, call **`POST /v1/expenses/:id/approve`** with body `{}` . The status becomes `approved` and the record can't be edited any more. Approving is allowed even if some items are left; show the user a warning first.
    - In a draft, an entry's `quantity` , `orderid` (adjusted) or `note` (consumed) may be left empty; they are listed in `incomplete` , and Approve answers **422** until they are filled.
    - `actions.approve` is `true` for the user who handles the order while the record is a draft.

Errors

- **422**, `details.order` : the order was cancelled before pickup (nothing to record), or it is not cancelled.
- **403**, the order is handled by the other city's team.
- **409**, a record for this order already exists.
- **422**, `details.items[0].dispositions[1].note` : consumed without "who".
- **422**, `details.items[0].dispositions` : entries are more than the quantity.
- **409**, calling submit on a cancelled-order record.

7. Working with expenses

7.1 Create: `POST /v1/expenses`

JSON body:

```
{
  "category": "PETROL",
  "expense_date": "2026-10-10",
  "notes": "Optional note",
  "client_request_id": "6f1c2b8e-0d1a-4f3e-9a51-3b2c7d9e0a11",
  "submit": true,
  "details": { "...": "see section 6" }
}
```

Field	Required	Notes
`category`	yes	`FOOD_PURCHASE` , `INTERCITY_LOGISTICS` , `DELIVERY` , `MISC` , `PETROL` , `TRAVEL` , `CANCELLED_ORDER`
`details`	yes	Category fields (section 6)

Field	Required	Notes
`expense_date`	no	Default now. No future dates; at most 7 days back
`notes`	no	Up to 1,000 characters
`client_request_id`	recommended	UUID, one per new expense form (section 3.5)
`submit`	no	`true` : send for approval straight away (deducts cash; everything must be filled in). Otherwise saved as **draft** (may be incomplete, see "Save draft" below)

Multipart (expense + photos in one call): `Content-Type: multipart/form-data`

- field **`data`** : the JSON body above, as a string;
- field **`files`** : up to 10 files in one call (repeat the field for each file). An expense can hold any number: add the rest with `POST /v1/expenses/:id/documents` while it is a draft.

Response **201**, `data` = the full expense (section 7.2). Possible extra fields:

- `warning` : e.g. default distance not available;
- on create + submit: `balance_after` (paise) and a `warning` if the balance went negative.

Documents are optional in every category.

7.2 The expense object: `GET /v1/expenses/:id`


```
{
  "status": "success",
  "data": {
    "expense": {
      "_id": "6ac990c0195df90f1c7635b4",
      "expense_no": "FP-2627-000001",
      "category": "FOOD_PURCHASE",
      "status": "submitted",
      "expense_date": "2026-10-10T06:41:28.219Z",
      "business_date": "2026-10-10",
      "source": "logistics_app",
      "amount": { "subtotal": 118000, "discount": 2000, "tax": 5800, "total": 121800 },
      "details": {
        "orders": [ "..." ],
        "vendor_city": { "_id": "...", "name": "KOLKATA" },
        "gst_applicable": true,
        "gst_rate_bp": 500,
        "items": [ "..." ],
        "total_weight_kg": 3.2
      },
      "order_refs": [
        { "order": "6ac33f67cd32c8f0c6301fa6", "orderid": "T2P-1790000101" },
        { "order": "6ac25d53cd32c8f0c62d7a74", "orderid": "T2P-1790000102" }
      ],
      "notes": "Bought at the counter",
      "documents": [
        {
```

```
        "_id": "6ac990c0195df90f1c7635be",
        "name": "bill.jpg",
        "mime": "image/jpeg",
        "size": 184220,
        "uploaded_at": "2026-10-10T06:41:30.303Z",
        "removed": false
      }
    ],
    "expense_for": {
      "_id": "605dc565d5eaba0008a50e4e",
      "name": "Ravi Kumar",
      "mobile": "9800000010",
      "user_type": "delivery_partner"
    },
    "submitted_by": { "...": "same shape" },
    "city": { "_id": "6040eae14a4b6c0008fe1aff", "name": "KOLKATA" },
    "office": { "_id": "60ef0e96d4473c18f88872f5", "name": "KOLKATA OFFICE" },
    "review": {
      "reviewed_by": null,
      "reviewed_at": null,
      "approval_note": "",
      "rejection_reason": ""
    },
    "ledger": { "state": "posted", "cycle": 1 },
    "flags": [],
    "incomplete": [],
    "submitted_at": "2026-10-10T06:41:31.374Z",
    "cancel_reason": "",
    "version": 3,
    "createdAt": "2026-10-10T06:41:28.257Z",
    "updatedAt": "2026-10-10T06:41:31.374Z"
  },
  "category_label": "Food Purchase",
  "actions": {
    "edit": false,
    "upload": false,
    "submit": false,
    "recall": true,
    "approve": false,
    "reject": false,
    "cancel": true,
    "delete": false
  },
  "history": [
    {
      "action": "created",
      "from_status": null,
      "to_status": "draft",
      "actor": { "name": "Ravi Kumar", "...": "..." },
```

```
        "source": "logistics_app",
        "note": "",
        "at": "2026-10-10T06:41:28.257Z"
      },
      { "action": "submitted", "from_status": "draft", "to_status": "submitted", "...": "..." }
    ],
    "ledger_entries": [
      {
        "txn_no": "CL-2627-000001",
        "type": "EXPENSE_DEBIT",
        "direction": "debit",
        "amount": 121800,
        "running_balance": 378200,
        "business_date": "2026-10-10"
      }
    ],
    "cash_balance": 378200
  }
}
```


Key fields for the UI:

Field	Use
`expense_no`	Show as the reference (FP-…, IL-…, DL-…, MS-…, PF-…, TR-…)
`status`	Status badge
`amount.total`	Amount in paise
`review.rejection_reason`	Show when `status` is `rejected`
`cancel_reason`	Show when `status` is `cancelled`
`actions`	Which buttons to show (section 4.1)
`history[]`	Timeline: created, submitted, rejected, resubmitted, …
`ledger.state`	`none` (not deducted), `posted` (deducted), `reversed` (put back). Rarely `pending` or `reversal_pending` for a moment while the server finishes the entry
`flags[]`	Warnings the office will see, e.g. possible duplicate. Optional to show
`incomplete[]`	Empty while complete. In a half-filled draft: the missing fields (`field` , `message`); show the "Incomplete" label and highlight them
`cash_balance`	The user's cash balance now

7.3 My expenses list: `GET /v1/expenses`

Query parameters (all optional):

Param	Example	Notes
`page`	`1`	Starts at 1

Param	Example	Notes
`limit`	`20`	Maximum 100
`status`	`submitted` or `draft,rejected`	Comma-separated
`category`	`PETROL`	Comma-separated
`from` , `to`	`2026-10-01`	By expense date (YYYY-MM-DD)
`q`	`T2P-1790000101`	Search expense number, order number, note
`sort`	`expense_date`	Also `amount` , `created` , `expense_no` , `status`
`dir`	`desc`	Or `asc`


```
{
  "status": "success",
  "data": [
    {
      "_id": "6ac990c0195df90f1c763660",
      "expense_no": "PF-2627-000001",
      "category": "PETROL",
      "category_label": "Petrol Filling",
      "status": "submitted",
      "expense_date": "2026-10-10T03:45:00.000Z",
      "amount": { "subtotal": 34050, "discount": 0, "tax": 0, "total": 34050 },
      "details": { "litres": 3.25 },
      "order_refs": [],
      "document_count": 1,
      "incomplete": [],
      "review": { "rejection_reason": "" },
      "version": 2
    }
  ],
  "totals": {
    "count": 6,
    "amount": 194625,
    "by_status": {
      "draft": { "count": 0, "amount": 0 },
      "submitted": { "count": 5, "amount": 146500 },
      "approved": { "count": 0, "amount": 0 },
      "rejected": { "count": 0, "amount": 0 },
      "cancelled": { "count": 1, "amount": 48125 }
    }
  },
  "pagination": { "page": 1, "limit": 20, "total": 6, "pages": 1 }
}
```


`totals.by_status` gives the counts for the status tabs. Show an **"Incomplete"** label on rows whose `incomplete` list is not empty. The list rows contain only summary fields; open `GET /v1/expenses/:id` for full details.

7.4 Edit: `PATCH /v1/expenses/:id`

Only allowed while the status is **draft** or **rejected** (`actions.edit` is true).

```
{
  "version": 4,
  "expense_date": "2026-10-09",
  "notes": "Corrected amount",
  "details": {
    "head_other": "Packing tape",
    "description": "2 rolls of tape",
    "reason": "Packing boxes for Delhi",
    "amount": "80"
  }
}
```

- **Send the complete `details` again**, not only the changed field.
- `expense_date` and `notes` are optional.
- **Food Purchase:**
  - send `items` for every line;
  - send `orders` only if the selection changed (if you leave `orders` out, the current orders are kept).
- Editing does **not** submit. After editing a rejected expense, call submit (7.5).
- An edit is always saved like a draft: fields may be left empty and `incomplete` is recalculated. Wrong values are still refused (422).
- **Food Purchase GST:** if you leave out `items[].gst_applicable`, a product in an edited purchase keeps its current GST and a new one has GST Yes at 5%.

7.5 Submit, move to draft, cancel

Call	Body	When
`POST /v1/expenses/:id/submit`	`{}` or `{ "version": 4 }`	Draft or rejected. Deducts cash. Response has `balance_after` and maybe `warning`. **422** while `incomplete` is not empty (the `errors` list names the missing fields)
`POST /v1/expenses/:id/recall`	`{}`	Submitted, before the office reviews it. Back to draft; cash is put back
`POST /v1/expenses/:id/cancel`	`{ "reason": "Entered twice" }`	Draft (reason optional), submitted or rejected (reason **required**, at least 3 characters). Cash is put back if it was deducted

Each returns the full expense as `data` (same as 7.2), plus `message`.

Submitting an incomplete draft:

```
{
  "status": "validation_error",
  "message": "This draft is not complete. Fill in the missing details, then try again.",
  "errors": [ { "field": "details.items[0].rate", "message": "items[0].rate is required" } ]
}
```

Delete (`DELETE /v1/expenses/:id`) exists only for the main admin in the admin panel; the app never gets it (`actions.delete` is always `false` in the app). A deleted expense disappears from lists and totals, any cash deducted for it is put back (an `EXPENSE_REVERSAL` entry), and its orders can be used again. In the cash ledger its entries show `expense_status: "deleted"` .

```
{
  "status": "success",
  "message": "Expense submitted",
  "data": { "...": "..." },
  "ledger_entry": "CL-2627-000001",
  "balance_after": 378200
}
```

```
{
  "status": "success",
  "message": "Expense submitted",
  "data": { "...": "..." },
  "ledger_entry": "CL-2627-000009",
  "balance_after": -72450,
  "warning": "Cash balance is now negative for this person."
}
```

7.6 Photos and documents

Call	Notes
`POST /v1/expenses/:id/documents`	Multipart, field **`files`** (1–10 files per call; call again for more). Only while draft or rejected. No limit on how many an expense can hold
`DELETE /v1/expenses/:id/documents/:docId`	Removes a document while draft or rejected
`GET /v1/expenses/:id/documents/:docId`	Returns the file. Add `?download=1` to download instead of view

- **Allowed:** JPG, PNG, WEBP, HEIC, PDF, up to 10 MB each. The type is checked from the file content, not the name.
- **Compression:** the server compresses photos (longest side 1600 px, JPEG) and removes photo metadata, including GPS, so upload the camera photo as it is. HEIC and PDF are stored unchanged. The stored file may therefore be `.jpg` even if you uploaded `.png` .
- **Viewing:** files are **private**. Load them with the `Authorization` header, e.g. an image loader with headers, or download to a temporary file. There are no public links.

7.7 Duplicate protection the app may see

Situation	Response
Same `client_request_id` sent again	200, the same expense
Order already in another active food purchase	409, with the order number and the FP number
Second delivery expense for the same order and side	409

Situation	Response
Same photo already attached to another expense, or same amount on the same day	Saved, but `flags[]` warns the office

8. My cash: `GET /v1/cash-ledger/me`

Query (optional):

Param	Notes
`from` , `to`	`YYYY-MM-DD` , the period for the summary and entries
`type`	One of the entry types below
`page`	Default 1
`limit`	Default 50, maximum 200


```
{
  "status": "success",
  "data": {
    "user": {
      "_id": "605dc565d5eaba0008a50e4e",
      "name": "Ravi Kumar",
      "mobile": "9800000010",
      "user_type": "delivery_partner"
    },
    "account": {
      "balance": 378200,
      "opening_set": true,
      "city": { "_id": "6040eae14a4b6c0008fe1aff", "name": "KOLKATA" }
    },
    "summary": {
      "opening_balance": 0,
      "received_from_head_office": 500000,
      "expenses_deducted": 121800,
      "closing_balance": 378200,
      "by_type": {
        "OPENING_BALANCE": { "amount": 0, "count": 0 },
        "CASH_RECEIVED_HO": { "amount": 500000, "count": 1 },
        "EXPENSE_DEBIT": { "amount": 121800, "count": 1 },
        "...": "..."
      }
    },
    "entries": [
      {
        "txn_no": "CL-2627-000002",
        "type": "EXPENSE_DEBIT",
        "type_label": "Expense",
```

```
        "direction": "debit",
        "amount": 121800,
        "running_balance": 378200,
        "entry_date": "2026-10-10T06:41:28.219Z",
        "business_date": "2026-10-10",
        "expense": "6ac990c0195df90f1c7635b4",
        "expense_no": "FP-2627-000001",
        "expense_status": "submitted",
        "note": "Food Purchase FP-2627-000001"
      },
      {
        "txn_no": "CL-2627-000001",
        "type": "CASH_RECEIVED_HO",
        "type_label": "Cash Received from Head Office",
        "direction": "credit",
        "amount": 500000,
        "running_balance": 500000,
        "business_date": "2026-10-09",
        "mode": "upi",
        "reference": "UTR 4521889",
        "note": ""
      }
    ],
    "pagination": { "page": 1, "limit": 50, "total": 2, "pages": 1 }
  }
}
```


How to show it

- `account.balance` is the **cash in hand now**. Show it in red if negative (the person has spent more than they received).
- Entries are newest first. `direction`: credit = money in (+), debit = money out (–). `running_balance` is the balance after that entry.
- For `EXPENSE_DEBIT` entries, show `expense_no` with a badge from `expense_status` (Pending approval, Approved, …), and tap to open the expense. `expense_status: "deleted"` means the office deleted it: show "Deleted" and don't open it.
- For `EXPENSE_REVERSAL`, `note` says why the money came back, e.g. `"MS-2627-000001 Rejected: Bill photo missing"`.

Entry type	Meaning	Direction
`OPENING_BALANCE`	Starting cash, entered by the office	credit
`CASH_RECEIVED_HO`	Cash sent by Head Office (`mode`: cash/bank/upi/other, `reference`)	credit
`EXPENSE_DEBIT`	An expense was submitted	debit
`EXPENSE_REVERSAL`	An expense was rejected, cancelled, deleted by the office or moved back to draft	credit

Entry type	Meaning	Direction
`ADJUSTMENT_CREDIT` / `ADJUSTMENT_DEBIT`	Correction by the office, with a reason in `note`	credit / debit
`DUTY_DEBIT`	Duty turned off: km × rate. `reference` has the order number on an order duty	debit
`DUTY_REVERSAL`	Late GPS points lowered a finished duty's km, so the difference came back	credit

8.1 Add money by screenshot: `POST /v1/ocr/payment-bill`

The office sends money by bank or UPI and shares the payment screenshot. The person who received the money uploads it here. The server reads the image and, if it is accepted, adds the amount to **the cash balance of the logged-in user**. There is nothing to type and no approval step.

Request: `multipart/form-data` with one image in a field named `file`. JPG, PNG, WEBP or HEIC, at most 10 MB. PDF is not accepted. Don't compress the image heavily, because the text must stay readable.

```
POST /v1/ocr/payment-bill
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9....
Content-Type: multipart/form-data; boundary=...
```

Timeout: usually a few seconds, but it can take longer when the reading service is busy. Show a loader and set the request timeout to **3 minutes**.

Success: HTTP 201


```
{
  "status": "success",
  "message": "Money added to your cash balance",
  "data": {
    "_id": "6ae5fa9ef8c51d802082d0e2",
    "ref_no": "991028211",
    "amount": 5000,
    "status": "Submitted Successfully",
    "payment_date": "2026-10-31T12:47:48.000Z",
    "paid_from": {
      "name": "CHARABUNI SERVICES PRIVATE LIMITED",
      "account_last4": "4497",
      "bank": "Axis Bank",
      "ifsc": null,
      "upi_id": null
    },
    "paid_to": {
      "name": "Harish Chourasia",
      "account_last4": "2978",
      "bank": "UNION BANK OF INDIA",
      "ifsc": "UBIN0537306",
      "upi_id": null
    },
```

```
    "crn": null,
    "ledger_entry": "CL-2627-000001",
    "balance_after": 500000
  }
}
```

> **Units differ in this response:** `amount` is in **rupees** (5000 = ₹5,000), and `balance_after` is in **paise** like the rest of this guide (500000 = ₹5,000).

Any field that isn't printed on the screenshot is `null`; Google Pay screenshots, for example, have `upi_id` and no IFSC. `payment_date` is `null` if the date couldn't be read.

What the app should do

- Show a confirmation with the amount, `ref_no` and `paid_from.name`, then call `GET /v1/cash-ledger/me` to refresh. The money appears as a `CASH_RECEIVED_HO` entry with `reference` set to the `ref_no`, and the screenshot is on the entry's `documents`.
- If the amount shown is wrong, the user should tell the office. Entries can't be edited; the office corrects them with an adjustment.
- Show the server's `message` for every error below. Don't write your own wording, and don't add money on the app side.

When it is refused

HTTP	When	What the user should do
422	Not a payment screenshot	Upload the right image
422	The sender is not Charabuni Services or Maa Traders	Nothing to retry: this payment is not from the company
422	The sender's name isn't visible	Upload a screenshot that shows the sender's name
422	The payment isn't shown as done (failed, pending, processing, initiated). Wording such as Successful, Completed, Paid, Processed or Transferred is accepted	Upload again once it has succeeded
422	The reference number or amount couldn't be read	Upload a clearer image
422	The payment is older than the date this feature started	Contact the office
422	No file, wrong file type, or larger than 10 MB. The file field must be named `file`	Choose another image
409	This reference number is already saved, by this user or anyone else	Nothing to retry: no money is added again
401	Token missing or expired	Log in again (section 2.2)
403	The account is not a logistics user	Not available to this account
429	Too many requests: more than 20 a minute for one user, or the service is at its limit	Wait a minute and try again
502, **503**	The reading service is busy or unavailable. No money was added	Offer **Try again**

Retrying is safe. One reference number can only add money once. If the connection drops and a retry returns 409, the first upload worked: refresh the balance instead of showing a failure.

> **The office sees refused uploads.** Every 409, and every 422 for a screenshot that was read (the first six 422 rows above), is logged with the user's name, so the T2P office can see who uploaded what. Don't resend a refused screenshot automatically.

Ledger entries are never edited or deleted; corrections are always new entries.

9. Push notifications

When the office **approves**, **rejects**, or **cancels an approved** expense, the server sends a Firebase push to the user's `device_token`. The app already registers that token with `POST /app/update-token`.

```
{
  "notification": {
    "title": "Expense rejected",
    "body": "Miscellaneous MS-2627-000001 ₹85.00 — Bill photo missing"
  },
  "data": { "type": "expense", "expense_id": "6ac990c0195df90f1c763700", "status": "rejected" }
}
```

On tap, open `GET /v1/expenses/{expense_id}`. Titles: "Expense approved", "Expense rejected", "Approved expense cancelled".

10. Endpoint reference (app)

Method	Path	Purpose
POST	`/app/logistic-login`	Request OTP (existing)
POST	`/app/verify-logistic`	Verify OTP, get `token`
GET	`/v1/expenses/meta`	Categories, form settings, my profile and cities
GET	`/v1/expenses/lookups/cities`	Cities
GET	`/v1/expenses/lookups/expense-heads`	Miscellaneous heads
GET	`/v1/expenses/lookups/food-purchase-orders?q=`	My orders waiting for a food purchase (delivery tomorrow / day after)
GET	`/v1/expenses/lookups/food-purchase-lines?orders=id,id`	Combined product lines for ticked orders
GET	`/v1/expenses/lookups/orders?q=`	My orders for a delivery expense (delivery today / tomorrow)
GET	`/v1/expenses/lookups/orders/:id/default-distance?side=`	Default distance for an order
GET	`/v1/expenses/lookups/cancelled-orders?q=`	Cancelled orders I have to record

Method	Path	Purpose
POST	`/v1/expenses`	Create (draft or submit), JSON or multipart
GET	`/v1/expenses`	My expenses (filters, totals, pages)
GET	`/v1/expenses/:id`	Expense details
PATCH	`/v1/expenses/:id`	Edit a draft or rejected expense
POST	`/v1/expenses/:id/submit`	Submit or resubmit
POST	`/v1/expenses/:id/recall`	Move back to draft
POST	`/v1/expenses/:id/cancel`	Cancel
POST	`/v1/expenses/:id/approve`	**Cancelled Orders only:** approve my record (final)
POST	`/v1/expenses/:id/documents`	Add photos/files
DELETE	`/v1/expenses/:id/documents/:docId`	Remove a photo/file
GET	`/v1/expenses/:id/documents/:docId`	View/download a photo/file
GET	`/v1/cash-ledger/me`	My cash balance and entries
POST	`/v1/ocr/payment-bill`	Upload a payment screenshot; adds the money to my cash balance

Office-only endpoints return **403** for app users: approve (except Cancelled Orders), reject, export, dashboard, other people's ledgers, Head Office cash entered by hand, adjustments, the log of refused screenshots.

11. Testing checklist

Please check these before release:

1. **Login:** OTP login stores the token; an expired or removed token sends the user back to login (401).
2. **Food Purchase:**
    - two orders with the same product show one line with the summed quantity;
    - the order quantity can't be typed;
    - a quantity change without a reason shows the field error;
    - every product needs a rate unless its final quantity is 0;
    - the totals shown after saving match the server.
3. **Food Purchase, orders:** using an order that is already in another purchase shows the 409 message; orders from two cities show an error.
4. **Intercity:** From City is pre-filled and locked.
5. **Delivery:** the default distance is shown read-only, and "not available" messages are displayed.
6. **Petrol:** two fillings on the same day both save.
7. **Travel:** "Other" requires the mode name.
8. **Retries:** turn off the network while submitting, turn it on and retry; you get the same expense, not a duplicate (`client_request_id`).
9. **Rejection:**
    - the rejected expense shows the reason;

    - edit → submit works;
    - the history shows "resubmitted";
    - the cash went back on rejection and was deducted again on resubmit.
10. **Cancel and move to draft:** the cash balance goes back up.
11. **Photos:** upload from the camera (large photo) and from the gallery, and a PDF; view them with the token; remove works only in draft or rejected.
12. **My cash:** balance, period summary and entries (newest first, with running balance); a negative balance is shown clearly.
13. **Push:** approve and reject from the admin panel; the notification opens the expense.
14. **Errors:** 422 errors appear next to the right fields; 409 reloads the expense; 429 waits and retries.
15. **Cancelled Orders:**
    - a pickup partner sees orders cancelled after pickup; a delivery partner sees orders cancelled from the origin airport onwards;
    - adjusted needs an order number, consumed needs "who";
    - Left updates as entries are added;
    - approve makes it final, with no edit afterwards;
    - the order disappears from the list once recorded.
16. **Delivery, partner on both sides:** the user can choose pickup or delivery.
17. **Save draft (v1.2):**
    - a half-filled form saves as a draft with the "Incomplete" label;
    - opening it highlights the missing fields; filling them and saving removes the label;
    - Submit on an incomplete draft shows the 422 message and field errors;
    - a wrong value (letters in a number) is refused even in a draft;
    - the cash balance does not change until Submit.
18. **GST (v1.3):** each row has its own switch, Yes by default. Switching one row to No removes GST from that row only; the on-screen totals match the server's. Editing a price without touching the switch keeps that row's GST.
19. **Order lists (v1.2):**
    - Food Purchase lists orders delivering tomorrow and the day after; Delivery lists today and tomorrow;
    - a filled but not approved order stays in the list (Food Purchase: shown with its expense and can't be ticked again);
    - after the office approves, the order disappears;
    - after the office deletes the expense, the order comes back (if its date is still in the window).
20. **Payment screenshot (v1.4):**
    - a Charabuni bank screenshot and a Maa Traders Google Pay screenshot are both accepted, and the balance goes up by the exact amount;
    - uploading the same screenshot again gives 409, and the balance doesn't change;
    - a payment from anyone else, and a photo that isn't a payment, show the server's message with no change to the balance;
    - the loader stays up during a slow upload; on 503, Try again works.

12. Current limitations

- **Delivery amount:** delivery expenses are saved with ₹0 until T2P decides the payment rule. The field will appear in `amount` later, with no app change needed.
- **Pickup default distance:** restaurant locations are not saved yet, so the default distance for **pickup** trips usually returns `km: null` with a message. Delivery-side distances work.
- **HEIC:** iPhone HEIC photos are stored without compression. To save data, the app may convert to JPEG before uploading (optional).
- **Approvals:** expenses are approved only in the admin panel, except Cancelled Orders, which the partner approves in the app.
- **Payment screenshots:** the server checks who sent the money, not who received it. Whoever uploads a screenshot first gets the money, so a screenshot must be uploaded by the person it was paid to.
- **Cancelled-order history:** who sees a cancelled order is worked out from the order's status history. If a status change was never recorded, the order may not appear in the list; report such orders to the T2P office.

For questions about this API, contact the T2P tech team.